

Entregable: Proyecto Final Sistema de Gestión Hospitalaria

Materia: Frontend 2

Profesor: Jossy Reinaldo Tello Landazuri

Institución: CESDE

Estudiante: José Alejandro Tangarife David

Fecha: 06 de Agosto de 2026

Parte 1: Documentación de Prompts (Uso de Inteligencia Artificial)

Durante el ciclo de desarrollo de este proyecto, se emplearon técnicas de *Prompt Engineering* interactuando con Inteligencia Artificial (LLM) para optimizar la construcción del código. A continuación, se detallan los 10 prompts técnicos utilizados, categorizados según las fases del proyecto exigidas en la rúbrica:

A. Diseño de Arquitectura

1. Estructura base y enrutamiento visual:

"Actúa como un Arquitecto de Software Frontend. Necesito estructurar un proyecto web sin backend llamado 'Sistema de Gestión Hospitalaria'. Los requisitos técnicos son HTML5, Bootstrap 5 y JavaScript Vanilla. Diseña la estructura de carpetas óptima y propón cómo dividir los archivos HTML para manejar de forma modular los tres roles exigidos (Administrador, Doctor, Paciente) asegurando la escalabilidad del código."

B. Estructura de Datos

2. Modelado relacional en LocalStorage:

"Diseña la estructura de datos en formato JSON para simular una base de datos relacional dentro de `localStorage`. Necesitamos tres entidades: 'Usuarios' (con nombre, email, contraseña y rol), 'Disponibilidad' (fechas y turnos médicos) y 'Citas_Medicas' (relacionando el email del paciente con la fecha, hora, especialidad y estado). Asegúrate de que las llaves foráneas implícitas permitan realizar filtros fácilmente en JavaScript."

C. Lógica de Negocio

3. Autenticación y Redirección Dinámica:

"Crea un script en JavaScript Vanilla para manejar la lógica de un login. La función debe interceptar el evento 'submit' de un formulario, prevenir la recarga, validar las credenciales contra el array de usuarios en `localStorage`, y crear un objeto de sesión llamado `usuario_activo`. Dependiendo del valor de la propiedad `rol` de este objeto, utiliza un switch para redirigir automáticamente (`window.location.href`) al panel correcto."

4. CRUD - Inserción de Citas:

"Escribe la lógica en JavaScript para que un 'Paciente' pueda agendar una cita. Construye una función que capture los inputs del DOM, genere un ID único irreplicable usando `Date.now()`, asigne el estado 'Pendiente' por defecto, e inserte el objeto en el array `citas_hospital` dentro del `localStorage` utilizando `JSON.stringify()`. Limpia el formulario al finalizar."

5. Gestión y mutación de Roles (Admin):

"Necesito una función iterativa para el panel del Administrador que lea el arreglo de usuarios y lo renderice en una tabla HTML. En la celda de acciones, inyecta un elemento `<select>` dinámico que muestre el rol actual de la persona, acompañado de un botón 'Guardar'. Escribe también la función que escuche el click del botón, encuentre al usuario por su email y actualice su rol permanentemente en la memoria del navegador."

6. Actualización de Estados (Doctor):

"Diseña una función de mutación de datos para la vista del Doctor. La interfaz mostrará botones de 'Marcar como Atendida' en cada fila de las citas pendientes. La función debe recibir el ID de la cita por parámetro, buscarla en el arreglo del `localStorage` mediante el método `.find()` o `.forEach()`, sobrescribir la propiedad 'estado' a 'Atendida', guardar los cambios y ejecutar una función `pintarTabla()` para refrescar el DOM en tiempo real."

D. Depuración (Debugging)

7. Manejo de excepciones en memoria vacía:

"Estoy obteniendo el error de consola `TypeError: Cannot read properties of null (reading 'push')` al intentar agregar un nuevo paciente usando `JSON.parse` cuando es la primera vez que se abre la aplicación. Explícame por qué ocurre este error al consultar una llave inexistente en el `LocalStorage` y reescribe la línea aplicando el operador de OR lógico (`|| []`) para inicializar un array vacío."

8. Blindaje de rutas y seguridad Frontend:

"He detectado una vulnerabilidad lógica: cualquier usuario puede escribir 'admin.html' en la URL y entrar a la vista protegida sin iniciar sesión. Escribe un bloque de código defensivo (Middleware de Frontend) que se ejecute en la primera línea del script, evalúe la existencia y el rol dentro del objeto `usuario_activo`, y expulse inmediatamente al usuario al `index.html` si carece de los permisos requeridos."

E. Mejora de Experiencia de Usuario (UX/UI)

9. Renderizado semántico de estados:

"Mejora la experiencia de lectura en las tablas de datos implementando jerarquía visual con Bootstrap 5. Escribe una función o condicional en JS que analice el estado de una cita médica (Pendiente, Confirmada, Atendida, Cancelada) y devuelva el HTML de un componente `<span>` (Badge), asignándole colores semánticos dinámicos (`warning`, `primary`, `success`, `danger`) según corresponda el texto."

10. Refactorización para mantenimiento:

"Refactoriza mi lógica de manipulación del DOM. Actualmente estoy usando `document.createElement`, `classList.add` y `appendChild` en un bucle enorme que hace el código ilegible. Reescribe esta función utilizando Template Literals (comillas invertidas) de ES6 y la propiedad `innerHTML`, para hacer la inyección del código HTML mucho más declarativa y fácil de mantener."

Parte 2: Reflexión Final y Análisis Crítico

El rol del Prompt Engineering y la IA en el Desarrollo de Software

Durante la construcción del Sistema de Gestión Hospitalaria, la integración de Inteligencia Artificial (IA) no se utilizó como un mero generador de código, sino como un asistente técnico de alto nivel. A través de este proyecto de la materia Frontend 2, quedó en evidencia que la calidad de la solución generada por la IA es directamente proporcional a la precisión de las instrucciones (Prompts) proporcionadas por el desarrollador.

El uso de la IA demostró un impacto altamente positivo en dos grandes frentes:

1. **Velocidad en el Boilerplate:** Permitió estructurar rápidamente los cimientos repetitivos del proyecto (maquetación de tablas, formularios y clases de Bootstrap).
2. **Eficiencia en la Depuración:** Al interactuar con el ecosistema asíncrono del navegador y el API de `localStorage`, errores como la manipulación de objetos nulos (`null`) en memoria fueron diagnosticados y resueltos en minutos gracias a la explicación de la IA sobre los operadores lógicos condicionales (`||` `[]`).

Sin embargo, el proceso también ratificó una regla de oro en la ingeniería de software actual: la IA complementa, pero no sustituye, el criterio del programador. Para poder

formular los prompts documentados en la sección anterior, fue necesario comprender primero la lógica de negocio subyacente (CRUD, persistencia de datos y protección de rutas). Si el desarrollador no conoce los conceptos estructurales, será incapaz de darle las restricciones correctas a la IA (como la prohibición estricta de usar backend impuesta en los requerimientos del proyecto).

En conclusión, dominar herramientas generativas eleva significativamente el perfil profesional. Al liberarnos de la memorización estricta de sintaxis, la IA nos permite enfocar nuestros esfuerzos cognitivos en lo verdaderamente esencial de nuestra carrera: diseñar arquitecturas escalables, construir interfaces intuitivas (UX/UI) y resolver problemas lógicos complejos que aporten valor real al usuario final.